HCL Interview Q

Company : HCL

Experience – 4 Years

Date : Jan, 2026

Q1. Explain OOPS concepts clearly with examples

1. Encapsulation

- **Concept:** Wrapping data (variables) and methods together into a single unit (class). It restricts direct access to fields using private and provides access via public methods (Getters/Setters).

```java
class Employee {
    private double salary; // Hidden data

    public void setSalary(double salary) {
        if(salary > 0) this.salary = salary; // Controlled access
    }
}
```

2. Inheritance

- **Concept:** One class acquires the properties and behaviors of another class using the extends keyword. It promotes code reusability.

```java
class Vehicle { // Parent
    void start() { System.out.println("Starting..."); }
}
class Car extends Vehicle { // Child
    void honk() { System.out.println("Honk!"); }
}
```

3. Polymorphism

- **Concept:** "One name, many forms." It allows objects to be treated as instances of their parent class but behave differently.
    - **Compile-time:** Method Overloading (Same name, different parameters).
    - **Runtime:** Method Overriding (Same method signature in child class).

```java
// Runtime Polymorphism
class Animal {
    void speak() { System.out.println("Animal noise"); }
}
class Dog extends Animal {
    @Override
    void speak() { System.out.println("Bark"); } // Specialized behavior
}
```

4. Abstraction

- **Concept:** Hiding implementation details and showing only functionality. Achieved using **Interfaces** or **Abstract Classes**.
- **Real-world:** You drive a car using the steering wheel without knowing how the engine combustion works.

```java
interface Payment {
    void pay(int amount); // Abstract method (No body)
}
class UPI implements Payment {
    public void pay(int amount) { System.out.println("Paid via UPI: " + amount);
}
```

Further Topics to Study

- **Diamond Problem:** Why Java doesn't support multiple inheritance with classes.
- **Composition vs. Inheritance:** Why "Has-A" is often better than "Is-A".
- **SOLID Principles:** Specifically the Liskov Substitution Principle.

Q2. Java fundamentals like class loading & memory allocation

1. Class Loading (Delegation Model) Java loads classes on demand, not all at once. It follows a hierarchy:

- **Bootstrap Loader:** Loads core Java classes (java.lang.*).
- **Extension Loader:** Loads external libraries (lib/ext).
- **Application Loader:** Loads **your** code and classpath jars.
- **Rule:** It always asks the parent loader first. If the parent can't find it, the child tries.

2. Memory Allocation JVM memory is split into two main parts:

- **Stack Memory:** Stores temporary variables and method calls. It is **Thread-Specific** (fast cleanup).
- **Heap Memory:** Stores actual **Objects** (created with new). It is **Shared** by all threads and managed by Garbage Collection.

Q3. Internal working of HashMap and hash collision handling

1. The Mechanics

- **Hashing:** The Key's hashCode() is calculated.
- **Index:** This hash is converted into an array index (Bucket).
- **Storage:** The Value is stored in that bucket.

2. Handling Collisions

- **The Problem:** Two keys land in the same bucket.
- **The Fix (Chaining):** Java stores both keys in a **Linked List** at that index.
- **Java 8+ Upgrade:** If the list grows > 8 nodes, it converts to a **Red-Black Tree**. This changes search speed from slow **O(n)** to fast **O(log n)**.

- Internally, a HashMap is an array of "buckets."
- When you insert a key-value pair, Java calculates the key's hashCode() to determine which index (bucket) it belongs to.
- If that bucket is empty, the data is stored there.
- However, if multiple keys end up at the same index (a "collision"), HashMap handles this by chaining them together in a **Linked List**.
- In Java 8 and later, there is a major performance upgrade: if a single bucket's list grows beyond 8 nodes, it automatically converts into a **Red-Black Tree**.
- This ensures that retrieving data remains fast (O(log n)) even when many collisions occur, rather than slowing down to a crawl (O(n)).

Q4. Types of exceptions and best practices

1. Types of Exceptions

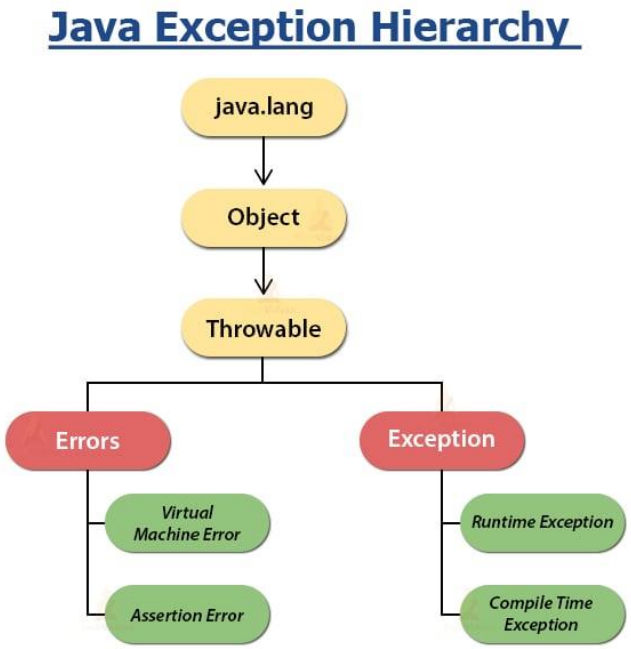
- **Checked Exception:** Checked at **compile-time**. You *must* handle them (try-catch or throws).
    - *Example:* IOException, SQLException.
- **Unchecked Exception (Runtime):** Occurs at **runtime**. Usually programming bugs.
    - *Example:* NullPointerException, IndexOutOfBoundsException.

2. Best Practices

- **Be Specific:** Catch specific exceptions (e.g., NumberFormatException) before generic Exception.
- **Never Swallow:** Don't leave a catch block empty. Log the error.
- **Clean Up:** Use **Try-with-Resources** (Java 7+) to automatically close connections/files.

Further Related Topics to Study

- **Custom Exceptions:** When to create your own (e.g., UserNotFoundException).
- **Difference between throw vs throws.**
- **Exception Hierarchy**



Q5. Basics of threads and multithreading

The Explanation

- **Thread:** A lightweight sub-process. It is the smallest unit of execution.
- **Multithreading:** The ability of a CPU to execute multiple threads concurrently (e.g., typing in Word while music plays in the background).
- **Main Thread:** Every Java program starts with one thread: the main thread.

1. Creating Threads (2 Ways)

- **Extend Thread Class:** Simple but limits inheritance (Java doesn't support multiple inheritance).
- **Implement Runnable Interface: Best Practice.** It keeps your class loose and allows it to extend other classes.

2. Thread Lifecycle (The 5 States) A thread goes through these stages:

1. **New:** Created but not started.
2. **Runnable:** Ready to run, waiting for CPU time.
3. **Running:** Currently executing instructions.
4. **Blocked/Waiting:** Paused (waiting for I/O or another thread).
5. **Terminated:** Finished execution

```
// Best Practice: Implement Runnable
Runnable task = () -> {
    System.out.println("Thread is running...");
};

Thread t1 = new Thread(task);
t1.start(); // Always call start(), NOT run()
```

Further Related Topics to Study

- **start() vs run():** Why calling run() directly doesn't start a new thread (it just runs as a normal method).
- **volatile keyword:** Preventing thread caching issues.
- **join() method:** Waiting for a thread to die.

Q6. Common real-time multithreading scenarios

1. Web Servers (The "Per-Request" Model)

- **Scenario:** Thousands of users hitting a website (e.g., Amazon) at the same time.
- **How it works:** The server (like Tomcat) maintains a **Thread Pool**. For every incoming HTTP request, it assigns a separate thread to handle that user. If 500 people click "Buy," 500 threads run simultaneously.

2. Background Tasks (Async Processing)

- **Scenario:** A user signs up and the system sends a Welcome Email.
- **How it works:** You don't make the user wait 5 seconds for the email to send. The main thread returns "Success" immediately, while a **worker thread** sends the email in the background.

3. Parallel Data Processing

- **Scenario:** Parsing a 10GB Excel file.
- **How it works:** Instead of reading row 1 to 1,000,000 sequentially, you split the file into 4 chunks. 4 threads process 250k rows each at the same time, finishing the job 4x faster.

Q7. Difference between ArrayList vs LinkedList – when to use which?

1. Structure & Memory

- **ArrayList:** Backed by a **Dynamic Array**. Elements are stored side-by-side in memory.
- **LinkedList:** Backed by a **Doubly Linked List**. Elements (Nodes) are scattered in memory, connected by pointers (Next/Prev).

2. Performance Comparison

- **Access (Get):**
    - **ArrayList: O(1) (Fast).** It has an index, so it jumps directly to the element.
    - **LinkedList: O(n) (Slow).** It must traverse from the head node to find an element.
- **Manipulation (Add/Remove):**
    - **ArrayList: Slow.** If you remove index 0, it must **shift** all subsequent elements left.
    - **LinkedList: Fast.** It simply changes the pointers of the neighboring nodes. No shifting needed.

3. When to use which?

- **Use ArrayList (Default):** For 99% of cases. It is cache-friendly and faster for searching/reading.

- **Use LinkedList:** Only if you are building a Queue/Stack or doing frequent additions/deletions at the **start or middle** of the list.

Q8. Difference between HashMap vs TreeMap

1. Ordering (The Main Difference)

- **HashMap:** Does **NOT** maintain any order. The elements may shuffle when the map resizes.
- **TreeMap:** Maintains a **Sorted Order**. It sorts keys based on their "Natural Ordering" (like A-Z, 1-10) or a custom Comparator.

2. Internal Structure & Performance

- **HashMap:** Uses a **Hash Table**.
    - *Speed:* **O(1)** (Constant time) for get() and put(). It is extremely fast.
- **TreeMap:** Uses a **Red-Black Tree** (Self-balancing binary search tree).
    - *Speed:* **O(log n)**. It is slower because it has to sort the key every time you insert it.

3. Null Keys

- **HashMap:** Allows **one** null key and multiple null values.
- **TreeMap:** Does **NOT** allow null keys (throws NullPointerException because it can't compare null to sort it).

Further Related Topics to Study

- **Comparator vs Comparable:** How to define custom sorting logic for a TreeMap.

Q9. Can we use a custom object as a key in HashMap? What needs to be overridden?

The Short Answer: YES. You can use any class (like Employee or Student) as a Key, but you **MUST** follow strict rules to prevent data loss.

1. What must be overridden? You must override **BOTH**:

- **hashCode():** Determines **which bucket** the key belongs to.
- **equals():** Determines if two keys in the same bucket are actually the **same object**.

2. The Contract (How it works)

- **Step 1:** HashMap calls key.hashCode() to find the bucket index.
- **Step 2:** If there is already data there (collision), it iterates through the list and calls key.equals(existingKey) to see if it's a duplicate or a new entry.
- **Failure Scenario:** If you don't override them, Java uses the default memory address. Even if two Employee objects have the same ID, Java will treat them as different keys.

3. The "Golden Rule": Immutability

- **Risk:** Custom keys should be **Immutable**.
- **Why?** If you modify a field inside the Key object *after* putting it into the Map, its hashCode() changes. When you try to get() it later, HashMap looks in the **wrong bucket**, and your data is lost forever (Memory Leak).

```java
class Student {
    final int id; // specific fields should be final (Immutable)

    @Override
    public int hashCode() { return Objects.hash(id); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return id == student.id;
    }
}
```

Q10. Scenario: Remove duplicate elements from a list while maintaining insertion order

The Solution

- Do NOT use a normal HashSet. It removes duplicates but **scrambles the order**.
- **The Fix:** Use a **LinkedHashSet**.
- It is a hybrid. It uses a Hash Table to ensure uniqueness, but maintains a **Doubly Linked List** running through all entries to remember the sequence in which they were inserted.

Two Ways to Do It

Way 1: The Classic Way (Constructor)

```
List<String> list = Arrays.asList("A", "B", "A", "C");

// Automatically removes duplicates & keeps order
Set<String> set = new LinkedHashSet<>(list);

// Convert back to list if needed
List<String> result = new ArrayList<>(set);
// Result: [A, B, C]
```

Way 2: The Java 8 Stream Way

```
List<String> result = list.stream()
                          .distinct() // Uses equals() to find duplicates
                          .collect(Collectors.toList());
```

Further Related Topics to Study

- **TreeSet:** Why it removes duplicates but sorts them (Natural Ordering), destroying insertion order.
- **CopyOnWriteArrayList:** How to handle this scenario safely in a multi-threaded environment.

Q11. Write a query to find the 2nd highest salary from an employee table

Method 1: The Quick Way (MySQL/PostgreSQL) Sort the salaries in descending order, skip the first one (OFFSET 1), and take the next one (LIMIT 1).

```
SELECT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Method 2: The Universal Way (Subquery) Find the maximum salary that is *less than* the absolute maximum salary.

```sql
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Further Related Topics to Study

- **Finding Nth Highest:** How to write a generic query to find the 5th, 10th, or Nth salary.

Q12. Difference between WHERE vs HAVING

1. WHERE Clause (Filters Rows)

- **Concept:** It filters individual records **before** any grouping happens. It looks at raw data in the table.
- **Restriction:** You cannot use functions like COUNT() or SUM() here.

Example: Pure WHERE *Scenario:* "Give me the list of all employees who live in 'New York'."

```sql
SELECT name, city
FROM Employees
WHERE city = 'New York';
-- Filters purely based on the column value 'city'.
```

2. HAVING Clause (Filters Groups)

- **Concept:** It filters summarized results **after** grouping happens. It looks at the result of an aggregation.
- **Requirement:** It is almost always used with GROUP BY.

Example: Pure HAVING *Scenario:* "Give me the list of departments that have **more than 10** employees."

```
SELECT dept_name, COUNT(*)
FROM Employees
GROUP BY dept_name
HAVING COUNT(*) > 10;
-- Filters based on the calculated value 'COUNT(*)'.
```

Further Related Topics to Study

- **Aggregation Functions:** SUM, AVG, MIN, MAX, COUNT.
- **Can you use both?**

Q13. Explain INNER JOIN vs LEFT JOIN with example

1. INNER JOIN (The "Exclusive" Join)

- **Concept:** Returns rows **only** when there is a match in **both** tables. If a record in Table A has no matching record in Table B, it is removed from the results.
- **Use Case:** "Show me only the active connections."

Example: *Scenario:* You have a Users table and an Orders table. You want to find **only** the users who have actually purchased something.

```
SELECT Users.Name, Orders.Product
FROM Users
INNER JOIN Orders ON Users.ID = Orders.UserID;

-- Result: Users who haven't ordered anything will NOT appear.
```

LEFT JOIN (The "Inclusive" Join)

- **Concept:** Returns **ALL** rows from the **Left** table, and the matching rows from the Right table. If there is no match, the Right side columns will be NULL.
- **Use Case:** "Show me everyone, and their data if it exists."

Example: *Scenario:* You have a Students table and a Grades table. You want a list of **all** students, even if they haven't taken the exam yet.

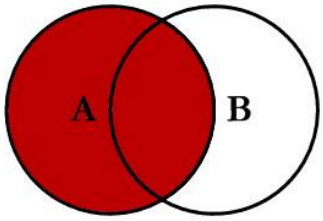
```
SELECT Students.Name, Grades.Score
FROM Students
LEFT JOIN Grades ON Students.ID = Grades.StudentID;

-- Result: All students appear.
-- If a student didn't take the exam, Score will be NULL.
```

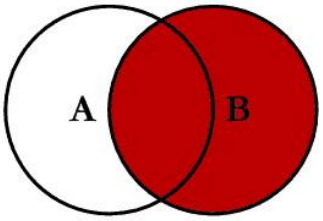
Further Related Topics to Study

- **RIGHT JOIN:** The exact opposite of Left Join (keeps all rows from the Right table).
- **FULL OUTER JOIN:** Returns everything from both tables, matching where possible and using NULLs where not.

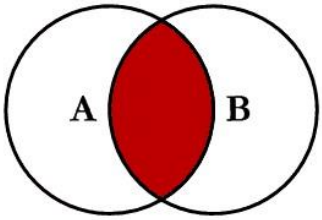
SQL JOINS



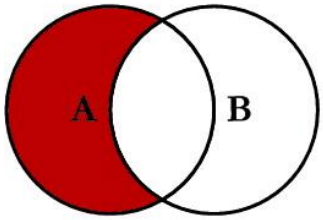

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



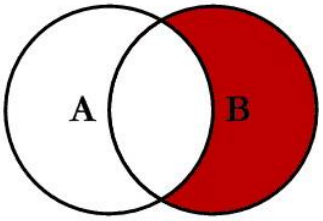

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



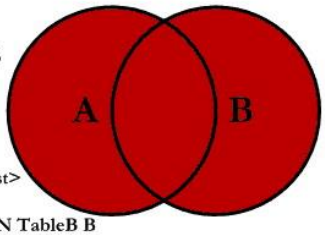

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



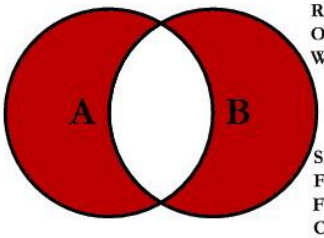

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008

Q14. Write a query using GROUP BY + HAVING

The Concept

- **GROUP BY:** Groups rows that have the same values into summary rows (e.g., "group all employees by department").
- **HAVING:** Filters those **groups** based on a condition (e.g., "only show groups where the total salary is > $100k").
- **Why not WHERE?** WHERE filters rows *before* grouping. You cannot use it to filter calculated results like SUM() or COUNT().

The Scenario *Problem:* "Find the total salary paid by each department, but **only** show departments that pay out more than $100,000 in total."

```sql
SELECT department, SUM(salary) as total_salary
FROM Employees
GROUP BY department
HAVING SUM(salary) > 100000;
```

Further Related Topics to Study

- **Aggregation Functions:** AVG(), MAX(), MIN(), COUNT().

Q15. What is the difference between Spring Framework vs Spring Boot?

- **Configuration: Spring** requires manual XML or Java setup. **Spring Boot** uses **Auto-Configuration** (scans classpath and sets up beans automatically).
- **Deployment: Spring** needs an external server (Tomcat/JBoss). **Spring Boot** has an **Embedded Server** (Tomcat inside the JAR).
- **Dependencies: Spring** requires manual version management. **Spring Boot** uses **Starters** (pre-bundled dependencies) to handle versions for you.

Q16. What is a Spring Bean and how is it created?

A **Spring Bean** is simply a Java Object that is created, configured, and managed by the **Spring IoC Container** (Inversion of Control).

The Difference

- **Normal Object:** You create it manually: User u = new User();
- **Spring Bean:** Spring creates it for you: @Autowired User u;

How is it created?

Spring scans your code at startup and creates beans using one of two main methods:

1. Using Class Annotation (Auto-Scan) Add @Component (or @Service, @Repository, @Controller) on top of your class.

```
@Component
public class MyService { ... }
```

2. Using Method Annotation (Manual Config) Add @Bean on a method inside a configuration class. Useful for third-party libraries.



```java
@Configuration
public class AppConfig {
    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

Q17. Explain Bean scopes (singleton, prototype, request, session)

Bean Scope determines the **lifecycle** and **visibility** of a bean. It answers: "How many instances of this object exist?"

1. Core Scopes (Available everywhere)

- **Singleton (Default):**
    - **Behavior:** Only **ONE** instance is created per Spring Container. It is cached and shared everywhere.
    - **Use Case:** Stateless services (e.g., UserService, DbConnection).
- **Prototype:**
    - **Behavior:** A **NEW** instance is created *every single time* you request it.
    - **Use Case:** Stateful objects (e.g., a non-thread-safe object).

2. Web-Aware Scopes (Only in Web Apps)

- **Request:** One instance per **single HTTP request**. It dies when the response is sent.
- **Session:** One instance per **User Session** (across multiple requests). It persists until the user logs out or the session times out.
- **Application:** One instance per ServletContext (Global to the web app).

Q18. How does Dependency Injection work in Spring?

Dependency Injection (DI) in Spring

The Concept

- **Manual Way:** You create dependencies yourself (Engine e = new Engine()).
- **Spring Way (DI):** You just declare you need it (@Autowired Engine e), and Spring **injects** it for you. "Don't call us, we'll call you."

How it Works? The **IoC Container** (Spring's Brain) does the heavy lifting:

1. **Scans:** Finds all classes marked with @Component or @Service.
2. **Creates:** Makes an instance of them (Beans).
3. **Injects:** Looks for @Autowired and plugs the matching Bean into your class.

The 3 Ways to Inject

1. **Constructor Injection (Best Practice):** Mandatory dependencies. You pass them in the constructor. Easy to test.
2. **Setter Injection:** Optional dependencies. Can be changed later.
3. **Field Injection (@Autowired on variable):** Easiest to write, but **bad practice** (hard to test, hides dependencies).

Q19. Difference between @Autowired vs constructor injection

@Autowired (Field) vs Constructor Injection

- **Field Injection (@Autowired on variable):** The "Lazy" way. It uses reflection to inject private fields directly.
- **Constructor Injection:** The **Professional (Best Practice)** way. It forces you to pass dependencies when creating the object.

Why Constructor Injection is BETTER:

1. **Immutability:** You can make fields final. Once set, they cannot change. Field injection *cannot* use final.
2. **Testing:** You can easily write Unit Tests with new Service(mockRepo). Field injection forces you to use reflection or Spring Test containers (slow).
3. **No Nulls:** The object is guaranteed to be fully initialized when created. Field injection risks NullPointerException if accessed too early.

Bad Practice (Field Injection)

```
@Service
public class UserService {
    @Autowired // Hides the dependency
    private UserRepository repo;
}
```

Best Practice (Constructor Injection)

```
@Service
public class UserService {
    private final UserRepository repo;

    // Explicit & Immutable
    public UserService(UserRepository repo) {
        this.repo = repo;
    }
}
```

Q20. Write a Java program to find duplicate elements in an array

```java
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 2, 7, 8, 8, 3};

        System.out.println("Duplicate elements are:");
        Set<Integer> seen = new HashSet<>();

        for (int num : arr) {
            // set.add() returns false if element exists
            if (!seen.add(num)) {
                System.out.print(num + " ");
            }
        }
    }
}
// Output: 2 8 3
```

Q21. Program to reverse a string without using inbuilt methods

```java
public class Main {
    public static void main(String[] args) {
        String str = "Automation";
        char[] chars = str.toCharArray(); // Convert to array

        int left = 0;
        int right = chars.length - 1;

        while (left < right) {
            // Swap characters
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;

            // Move pointers
            left++;
            right--;
        }

        String reversed = new String(chars);
        System.out.println(reversed);
    }
}
```